VALOR

Write with clarity. Validate before translation.

VALOR is a statically typed programming language with a source-to-source transpiler that translates VALOR programs into readable JavaScript. The generated JavaScript can then be executed using Node.js.

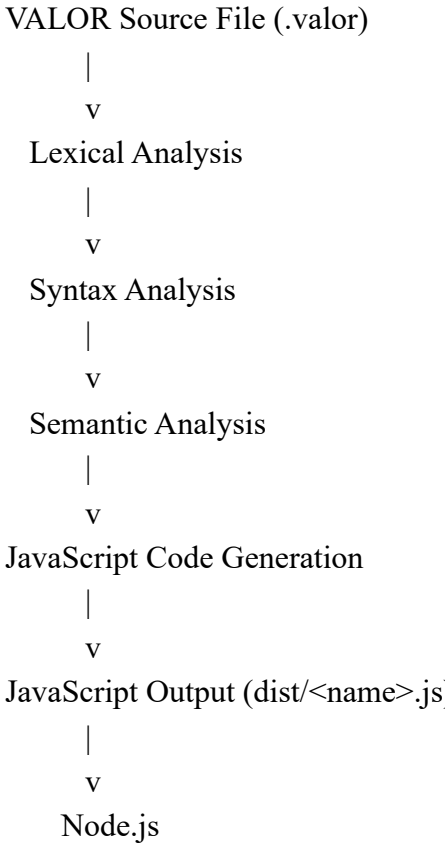
Overview

VALOR provides a simple, readable syntax while enforcing type rules before code generation.

The transpilation process has four stages:

1. **Lexical Analysis** converts source code into tokens.
2. **Syntax Analysis** checks the token sequence against VALOR's grammar and builds an Abstract Syntax Tree (AST).
3. **Semantic Analysis** checks declarations, types, and other language rules.
4. **JavaScript Code Generation** converts the validated AST into JavaScript.

A VALOR program is translated only after it passes all of these stages.



Language Features

- Explicit variable type annotations

- Basic types: number, string, and boolean
- Variable declarations using let, and assignment
- Arithmetic expressions (+, -, *, /, %), comparison expressions (==, !=, <, >, <=, >=), and logical expressions (&&, ||, !)
- Conditional statements using if and else
- Repetition using while
- Function declarations (fn) and return statements, including recursion
- Output statements using reveal(...)
- Single-line comments beginning with // and multi-line comments between /* and */

Not included in this version: for loops, lists, nested functions, and type inference.

Example

```
let studentName: string = "Reimar";
let score: number = 92;
let passingScore: number = 75;
let passed: boolean = score >= passingScore;

reveal("=== VALOR GRADE EVALUATOR ===");
reveal(studentName);
reveal(score);

if (passed) {
   reveal("Result: PASSED");
} else {
   reveal("Result: FAILED");
}
```

The reveal(...) statement is VALOR's output instruction. It is translated into JavaScript's console.log(...).

Type Rules

VALOR uses explicit type annotations. A variable declaration specifies its type:

```
let age: number = 21;
let name: string = "Alex";
let isEnrolled: boolean = true;
```

The initializer must be compatible with the declared type, and VALOR does not infer types automatically.

The + operator supports numeric addition (number + number) and string concatenation (string + string). Mixing a number and a string with + is rejected by semantic analysis.

Requirements

- **Node.js** (v24.15.0 was used for development). Required to run the VALOR transpiler and execute the generated JavaScript.
- **Visual Studio Code** or another code editor (recommended) for editing .valor source files.

There are no external npm dependencies, so npm install is not needed.

Check that Node.js is available:

node --version

Project Structure

valor/
├── src/
│ ├── main.js
│ ├── lexer.js
│ ├── parser.js
│ ├── semantics.js
│ ├── codegen.js
│ └── test_lexer.js
├── examples/
│ ├── hello.valor
│ └── signature.valor
├── dist/
│ └── (generated JavaScript files)
├── docs/
├── package.json
└── README.md

Getting Started

1. Open the project

Open the valor project folder in Visual Studio Code, then open the integrated terminal (**Terminal → New Terminal**). Make sure the terminal is in the project root, the folder that contains src, examples, and package.json. If necessary:

cd path/to/valor

2. Run the transpiler

node src/main.js

The main program reads the source file configured in src/main.js, runs it through the four transpilation stages, and writes the generated JavaScript file to dist/. For example, if src/main.js is configured to read examples/hello.valor, it writes dist/hello.js.

3. Execute the generated JavaScript

node dist/hello.js

Use the file name that was generated in your dist/ folder.

Creating and Running Another VALOR Program

1. Create a new file with the .valor extension inside examples/, for example examples/my_program.valor.
2. Write a program using VALOR syntax.
3. In src/main.js, change the source path to examples/my_program.valor and the output path to dist/my_program.js.
4. Run the transpiler:
5. Execute the generated JavaScript:

Error Handling

The transpiler reports an error and does not generate JavaScript when a stage cannot process the program:

- **Lexical errors:** an unrecognized character or invalid token.
- **Syntax errors:** a token sequence that does not follow the language grammar.
- **Semantic errors:** a type mismatch or another violation of a language rule, such as using number + string with the + operator.

Correct the error in the VALOR source and run the transpiler again.

Testing

The repository includes a lexer test script, src/test_lexer.js. Run it from the project root:

node src/test_lexer.js

No test runner is configured. A reference interpreter and an automated differential test suite are planned as future work.

Author

Reimar Escueta Master of Science in Computer Science Technological Institute of the Philippines, Manila